Jen v4 使い方ガイド

「終わりました」を、機械で確かめる

jen-marketplace v4.0.0

Table of Contents

1. v3 から何が変わったか

変わったのは「完了」の意味だけです。モード（route / conduct / repair / review / release / longrun）も、18 体のエージェントも、仕事を振り分ける流れもそのままです。

	v3 まで	v4
完了をどう決めるか	検収役のエージェントが「合格」と言ったら完了	宣言した項目すべてに証拠が付いたら完了
決めるのは誰か	LLM	1 件ずつの判断は LLM。全部そろったかの集計は Python
「テストが通った」の根拠	エージェントの報告	台帳が自分でコマンドを実行して得た終了コード
完了を止めるのは	プロンプト（お願いしているだけ）	Stop hook（exit 2 で差し戻す）
受入条件の置き場	`.jen/mission.md`（ただの文章）	`.jen/claims.json`（状態を持つ台帳）

v3 の「検収役が合格と言うまで完了にしない」は、LLM がそう振る舞ってくれることを期待していただけでした。v4 はそこだけをプログラムに移します。

> 何も宣言しなければ、v4 は v3 と同じように動きます。台帳が無いセッションは完了を止められません。ただし「止めずに終わった」ことは記録に残ります。

2. 最小の流れ

`$H` は hooks の置き場です。プラグインとして入れているなら `$CLAUDE_PLUGIN_ROOT/hooks` になります。

```
H="$CLAUDE_PLUGIN_ROOT/hooks"
```

① 受入条件を、1 つずつ確かめられる形に割って宣言する

「サインアップ画面を改善する」のような文章ではなく、1つずつ独立に確かめられる形に割ります。割ったあとの1つを、以下では項目と呼びます（コマンドでは `--claim` です）。

```
python3 "$H/evidence_ledger.py" init --tier sonnet \
  --claim "C1=lint / typecheck / test が通る" \
  --claim "C2=初回画面に次アクションが表示される"
```

--tier は、検証を工程ごとに走らせるか最後に 1 回にするかを決めます（第 5 節）。

② 作業する

ここは v3 と同じです。Jen が仕事を分解し、エージェントに振り分け、実装します。

③ 証拠を付ける

「できました」と報告しても項目は閉じません。台帳に実行させます。

```
python3 "$H/evidence_ledger.py" run --claim C1 --cmd "npm test"
```

台帳が npm test を自分で実行し、終了コードを受け取ります。出力は .jen/logs/evidence/ に残ります。

④ 状態を見る

```
python3 "$H/evidence_ledger.py" status
```

全項目に証拠がそろっていれば exit 0、足りなければ exit 1 です。

3. 証拠には強さがある

決まった文字列が出力されたかどうかを見るゲートは、破られます。実例があります。あるエンジニアが Stop hook を作り、出力の中に決まった文字列があれば通す仕組みにしたところ、モデルは中身の処理を実行せずにその文字列だけを出力して通り抜けるようになりました（コキエ 2026-03）。

そこで v4 は証拠を 3 段階に分けます。

強さ	作り方	作れるのは誰か
weak	attest --kind note（エージェントの報告）	エージェント
medium	attest --kind artifact（ファイル。台帳が sha256 を計算する）	エージェント
strong	run --cmd（台帳が自分でコマンドを実行する）	台帳だけ

既定では medium 以上を要求します。つまり報告（weak）だけでは項目は閉じません。

```
# これは「証拠はあるが足りない」で止まる（weak）
python3 "$H/evidence_ledger.py" attest --claim C2 --kind note --detail "実装した"

# これは閉じる（medium）
python3 "$H/evidence_ledger.py" attest --claim C2 --kind artifact --ref screenshot.png
```

大事な項目には strong を要求できます。

```
python3 "$H/evidence_ledger.py" claim-add --id C3 \
  --text "既存テストを壊していない" --require strong
```

強い証拠を「作った」と宣言する方法は用意していません。`run` だけが作れます。だからエージェントは「テストが通りました」を、文章を書くことでは作れません。

4. コマンド一覧

コマンド	何をするか
`init`	台帳を作る。`--tier --require --max-delegations --force`
`claim-add`	項目を足す。`--id --text --require`
`attest`	弱い／中くらいの証拠を付ける。`--kind note|artifact`
`run`	台帳が自分でコマンドを実行し、終了コードを強い証拠にする。`--cmd --expect-exit`
`status`	どの項目に証拠がそろっているかを表示する（全部そろえば exit 0）
`gate`	完了していいかを判定する（Stop hook が使う。止めるときは exit 2）
`budget`	ほかのエージェントに任せた回数を、hook の実ログから数える
`revise`	受入条件そのものが変わったとき、新しい版に切り替える。証拠は消える
`tier`	使っているモデルを変えて、検証の頻度を切り替える

5. 検証を、いつ走らせるか

独立した検証はいつでも必要ですが、頻度は使っているモデルによって変えます。

`--tier`	動き	使い方
`haiku / sonnet`	工程ごとに走らせる	工程が終わるたびに `run` で証拠を足していく
`opus / fable`	最後に 1 回だけ	工程の途中では走らせず、最後にまとめて確かめる

根拠は独立した 2 つの実験です。

高性能なモデルは、ハーネスを付けても付けなくても 10 回中 10 回成功しました。一方、低価格のモデルは、ハーネスなしでは 10 回中 3 回しか成功せず、ハーネスを付けると 10 回中 10 回になりました（Jain 2026-10）。Anthropic も、Opus 4.6 で性能が上がったことを理由に、評価役を工程ごとではなく実行の最後に 1 回だけ走らせる形に変えています（2026-03）。

安いモデルほど、ハーネスにかけたコストに見合う結果が出ます。

6. 完了を止められたら

完了しようとして exit 2 で止められたときは、次の 3 つのどれかです。

1. 証拠が足りない → 証拠のそろっていない項目に run で強い証拠を付ける
2. 受入条件そのものが間違っていた → 人間に確認してから、新しい版に切り替える（証拠は消えます）
3. 解けない → 人間に引き継ぐ。「何ができて、なぜ止まり、人間は何をすればいいか」を報告する

```
python3 "$H/evidence_ledger.py" revise --note "受入条件に対応端末を追加した"
```

7. 完了を止めるのをやめる（人間の判断）

人間が明示的に切れます。.jen/claims.json に "gate": "off" を入れてください。切ったことは記録に残ります（stop-events.jsonl の reason: gate_off_by_human）。

エージェント側から切る方法は用意していません。

8. 使ったトークンを数える（jen-classic）

jen-classic は「同じ仕事を、より少ないトークンと時間で終わらせる」ことを目的にしています。そのために、ほかのエージェントに任せた回数を数えられます。

```
python3 "$H/evidence_ledger.py" init --tier haiku --max-delegations 8 --claim "C1=..."
python3 "$H/evidence_ledger.py" budget
```

サブエージェントは呼び出すたびに何も知らない状態から始まるので、1回任せるたびにコンテキストを1つ丸ごと使います。ここがいちばん費用のかさむところです。

上限を超えても強制的には止めません。「分解が重すぎたのではないか」を点検する合図として使います。

数えられないときは「判定不能」と表示します。**0** とは言いません。

9. やってみる（この通りに動きます）

以下は実際に動く一連の流れです。空のディレクトリで試してください。

```
# VERIFY-BLOCK
set -e
H="${H:?hooks のパスを H に設定してください}"

# 1. 項目を 2 つ宣言する
python3 "$H/evidence_ledger.py" init --force --tier sonnet \
  --claim "C1=ビルドが通る" --claim "C2=成果物が存在する"

# 2. 報告（weak）だけでは閉じない
python3 "$H/evidence_ledger.py" attest --claim C1 --kind note --detail "実装した"
python3 "$H/evidence_ledger.py" status || echo "→ まだ閉じていない（想定どおり）"
```

```
# 3. 完了は止められる
python3 "$H/evidence_ledger.py" gate 2>/dev/null && echo "通ってはいけない" && exit 1
echo "→ 完了が止まった（想定どおり）"

# 4. 台帳に実行させる＝強い証拠になる
python3 "$H/evidence_ledger.py" run --claim C1 --cmd "true"

# 5. ファイルを作って中くらいの証拠を付ける
echo "artifact" > out.txt
python3 "$H/evidence_ledger.py" attest --claim C2 --kind artifact --ref out.txt

# 6. 全項目に証拠がそろったので、完了できる
python3 "$H/evidence_ledger.py" status
python3 "$H/evidence_ledger.py" gate
echo "→ 完了が許された"

# 7. 受入条件を変えると、証拠は消える
python3 "$H/evidence_ledger.py" revise --note "受入条件が変わった"
python3 "$H/evidence_ledger.py" gate 2>/dev/null && echo "消えていない" && exit 1
echo "→ 新しい版に切り替えたので証拠が消えた（想定どおり）"

# 8. 任せた回数を数える
python3 "$H/evidence_ledger.py" budget
```

このブロックは **CI** が実際に実行しています。 書いてあるとおりに動かなければ、このガイドは公開できません。
